

IMCS UNIVERSITY OF SINDH

Human-Computer Interaction (HCI) & Computer Graphics (CG)

**Lab 1 Report: Display Density Metrics & Image Array Mechanics in
NumPy**

Student Name: [Hammad Ahmed](#)

Roll Number / Seat Number: [2k24/CSE/62](#)

Course: [HCI & Computer Graphics](#)

Department: [Department of Computer Science](#)

1. Task 1: Display Pixel Density (PPI/DPI) Calculator

Source Code

```
1  import math
2
3  # User input
4  w_px = int(input("Enter horizontal resolution (pixels): "))
5  h_px = int(input("Enter vertical resolution (pixels): "))
6  d_inches = float(input("Enter physical diagonal size (inches): "))
7
8  # Calculations
9  total_pixels = w_px * h_px
10 common_divisor = math.gcd(w_px, h_px)
11 aspect_w = w_px // common_divisor
12 aspect_h = h_px // common_divisor
13
14 diagonal_pixels = math.sqrt(w_px**2 + h_px**2)
15 dpi = diagonal_pixels / d_inches
16
17 # Density classification
18 if dpi < 100:
19     category = "Low Density (Standard Monitor)"
20 elif 100 <= dpi <= 200:
21     category = "Medium Density (HD Display)"
22 else:
23     category = "High Density (Retina / Mobile)"
24
25 # Display results
26 print("\n--- DISPLAY METRICS ANALYSIS ---")
27 print(f"Total Pixel Count: {total_pixels:,} pixels")
28 print(f"Aspect Ratio:      {aspect_w}:{aspect_h}")
29 print(f"Calculated DPI:      {dpi:.2f} DPI")
30 print(f"Density Category:  {category}")
31
```

Test Case 1: Desktop Monitor (1920 × 1080, 24")

- Horizontal Resolution: 1920
- Vertical Resolution: 1080
- Diagonal Size: 24
- Total Pixels: 2,073,600 pixels

- **Aspect Ratio:** 16:9
- **Calculated DPI:** 91.79 DPI
- **Density Classification:** Low Density (Standard Monitor)

```
Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 24

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count: 2,073,600 pixels
Aspect Ratio:      16:9
Calculated DPI:    91.79 DPI
Density Category:  Low Density (Standard Monitor)
```

Test Case 2: Smartphone Screen (1170 × 2532, 6.1")

- **Horizontal Resolution:** 1170
- **Vertical Resolution:** 2532
- **Diagonal Size:** 6.1
- **Total Pixels:** 2,962,440 pixels
- **Aspect Ratio:** 195:422
- **Calculated DPI:** 457.19 DPI
- **Density Classification:** High Density (Retina / Mobile)

```
Enter horizontal resolution (pixels): 1170
Enter vertical resolution (pixels): 2532
Enter physical diagonal size (inches): 6.1

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count: 2,962,440 pixels
Aspect Ratio:      195:422
Calculated DPI:    457.25 DPI
Density Category:  High Density (Retina / Mobile)
```

2. Task 2: Synthetic Image Matrix Creation

Source Code

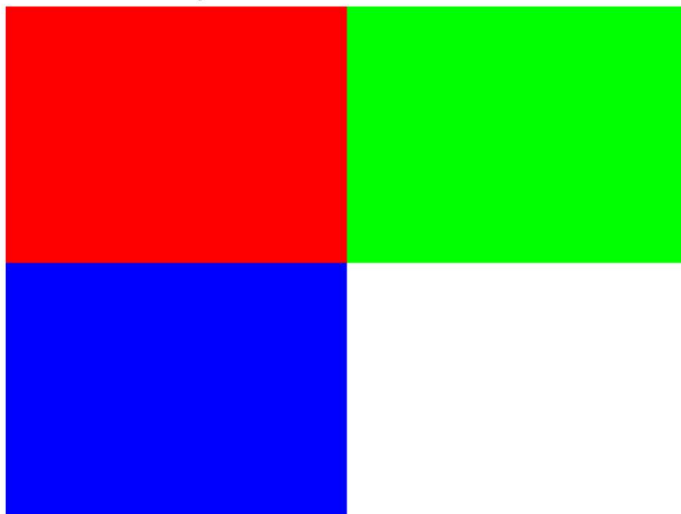
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 img_matrix = np.zeros((300, 400, 3), dtype=np.uint8)
5
6 mid_h, mid_w = 300 // 2, 400 // 2
7
8
9 img_matrix[:mid_h, :mid_w] = [255, 0, 0]
10
11 img_matrix[:mid_h, mid_w:] = [0, 255, 0]
12
13 img_matrix[mid_h:, :mid_w] = [0, 0, 255]
14
15 img_matrix[mid_h:, mid_w:] = [255, 255, 255]
16
17 print("--- SYNTHETIC MATRIX METRICS ---")
18 print(f"Array Shape (H, W, C): {img_matrix.shape}")
19 print(f"Data Type: {img_matrix.dtype}")
20 print(f"Total Elements: {img_matrix.size:,} values")
21 print(f"Memory Footprint: {img_matrix.nbytes:,} bytes ({img_matrix.nbytes / 1024:.2f} KB)")
22
23 plt.imshow(img_matrix)
24 plt.title("Synthetic 4-Quadrant Matrix")
25 plt.axis("off")
26 plt.show()
```

Execution Metrics Output

```
--- SYNTHETIC MATRIX METRICS ---
Array Shape (H, W, C): (300, 400, 3)
Data Type: uint8
Total Elements: 360,000 values
Memory Footprint: 360,000 bytes (351.56 KB)
```

Figure 1

Synthetic 4-Quadrant Matrix



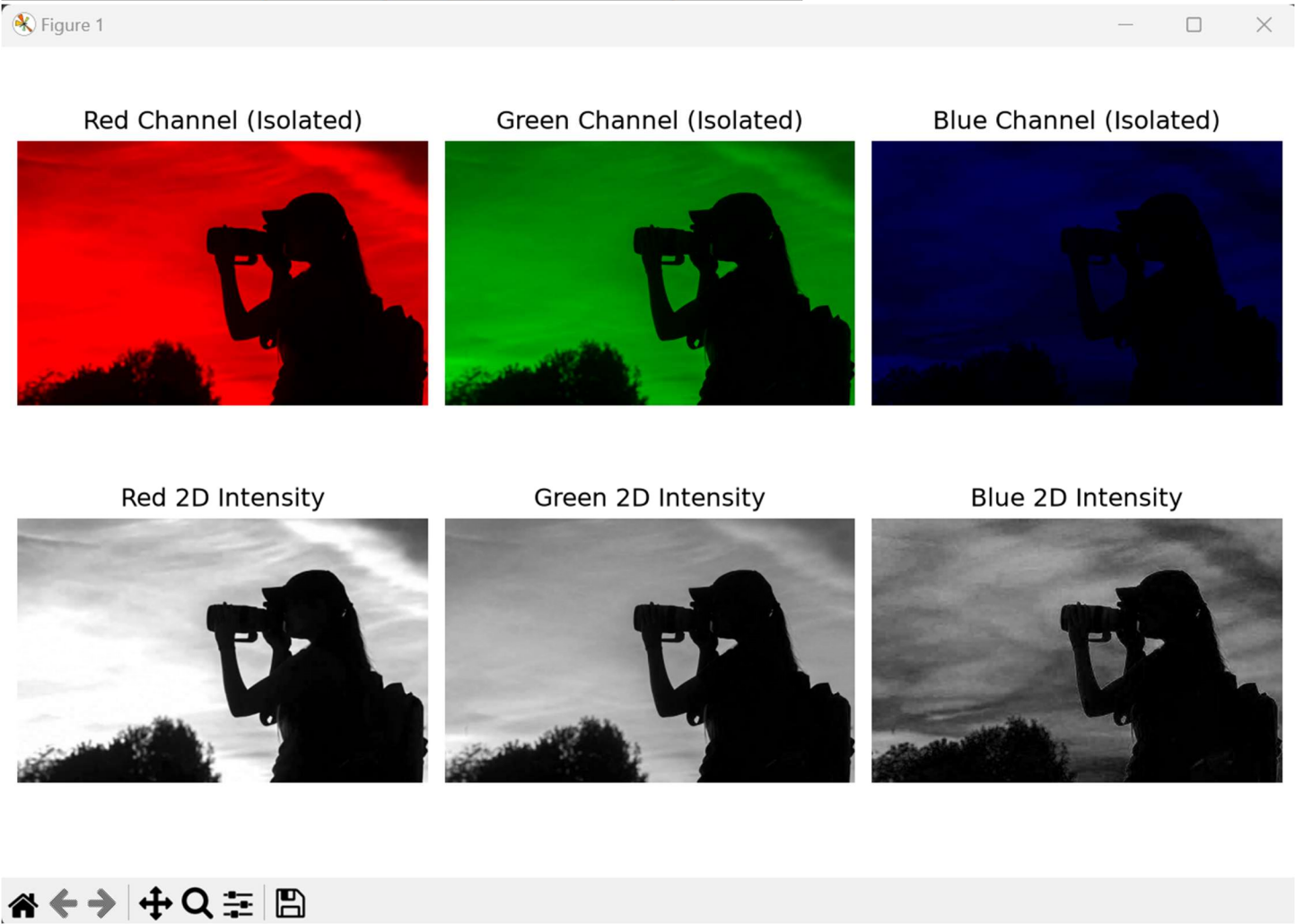
3. Task 3: Channel Slicing & Isolation

Source Code

```
1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from PIL import Image
5
6 # Automatically resolve the correct path to sample.jpg
7 script_dir = os.path.dirname(os.path.abspath(__file__))
8 image_path = os.path.join(script_dir, 'sample.jpg')
9
10 # 1. Load image and convert to NumPy array
11 raw_img = Image.open(image_path)
12 img = np.array(raw_img)
13
14 # 2. Extract 2D intensity grids via Axis 2 slicing[cite: 1]
15 r_2d = img[:, :, 0]
16 g_2d = img[:, :, 1]
17 b_2d = img[:, :, 2]
18
19 # 3. Construct 3D single-channel isolation matrices[cite: 1]
20 red_only = np.zeros_like(img)
21 red_only[:, :, 0] = r_2d
22
23 green_only = np.zeros_like(img)
24 green_only[:, :, 1] = g_2d
25
26 blue_only = np.zeros_like(img)
27 blue_only[:, :, 2] = b_2d
28
29 # Print metrics summary[cite: 1]
30 print("--- CHANNEL EXTRACTION SUMMARY ---")
31 print(f"Original Image Shape: {img.shape}")
32 print(f"Red Channel 2D Shape: {r_2d.shape} | Mean Intensity: {r_2d.mean():.2f}")
33 print(f"Green Channel 2D Shape: {g_2d.shape} | Mean Intensity: {g_2d.mean():.2f}")
34 print(f"Blue Channel 2D Shape: {b_2d.shape} | Mean Intensity: {b_2d.mean():.2f}")
35
36 # 4. Render 2x3 Subplot Grid[cite: 1]
37 fig, axes = plt.subplots(2, 3, figsize=(12, 7))
38
39 # Top row: Color isolation[cite: 1]
40 axes[0, 0].imshow(red_only)
41 axes[0, 0].set_title("Red Channel (Isolated)")
42 axes[0, 1].imshow(green_only)
43 axes[0, 1].set_title("Green Channel (Isolated)")
44 axes[0, 2].imshow(blue_only)
45 axes[0, 2].set_title("Blue Channel (Isolated)")
46
47 # Bottom row: Grayscale intensity maps[cite: 1]
48 axes[1, 0].imshow(r_2d, cmap='gray')
49 axes[1, 0].set_title("Red 2D Intensity")
50 axes[1, 1].imshow(g_2d, cmap='gray')
51 axes[1, 1].set_title("Green 2D Intensity")
52 axes[1, 2].imshow(b_2d, cmap='gray')
53 axes[1, 2].set_title("Blue 2D Intensity")
54
55 for ax in axes.ravel():
56     ax.axis("off")
57
58 plt.tight_layout()
59 plt.show()
```

Visual Output

```
--- CHANNEL EXTRACTION SUMMARY ---
Original Image Shape:  (444, 691, 3)
Red Channel 2D Shape:  (444, 691) | Mean Intensity: 150.80
Green Channel 2D Shape: (444, 691) | Mean Intensity: 91.86
Blue Channel 2D Shape: (444, 691) | Mean Intensity: 38.15
```



4. Task 4: Spatial Downsampling & Pixelation via Striding

Source Code

```
1  import os
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from PIL import Image
5
6  # Automatically resolve the correct path to sample.jpg
7  script_dir = os.path.dirname(os.path.abspath(__file__))
8  image_path = os.path.join(script_dir, 'sample.jpg')
9
10 # 1. Load image and convert to NumPy array
11 raw_img = Image.open(image_path)
12 img = np.array(raw_img)
13
14 N = 8
15
16 # 2. Downsample using strided slicing along axis 0 and axis 1
17 downsampled = img[::N, ::N, :]
18
19 # 3. Re-expand using np.repeat across axes 0 and 1
20 pixelated = np.repeat(np.repeat(downsampled, N, axis=0), N, axis=1)
21
22 # Crop re-expanded array to match exact dimensions if not cleanly divisible by N
23 pixelated = pixelated[:img.shape[0], :img.shape[1], :]
24
25 # 4. Memory & dimension reduction metrics
26 dim_reduction = (1 - (downsampled.shape[0] / img.shape[0])) * 100
27 mem_savings = (1 - (downsampled.nbytes / img.nbytes)) * 100
28
29 print(f"--- DOWNSAMPLING ANALYSIS (N={N}) ---")
30 print(f"Original Shape:      {img.shape} | Memory: {img.nbytes:,} bytes")
31 print(f"Downsampled Shape:     {downsampled.shape} | Memory: {downsampled.nbytes:,} bytes")
32 print(f"Re-expanded Shape:     {pixelated.shape} | Visual: Blocky Pixelation")
33 print(f"Dimension Reduction:    {dim_reduction:.2f}% reduction per axis")
34 print(f"Memory Savings:        {mem_savings:.2f}% data reduction")
35
36 # 5. Visual comparison
37 fig, ax = plt.subplots(1, 2, figsize=(10, 5))
38 ax[0].imshow(img)
39 ax[0].set_title("Original Image")
40 ax[0].axis("off")
41
42 ax[1].imshow(pixelated)
43 ax[1].set_title(f"Pixelated (Downsampled N={N} & Re-expanded)")
44 ax[1].axis("off")
45
46 plt.tight_layout()
47 plt.show()
```

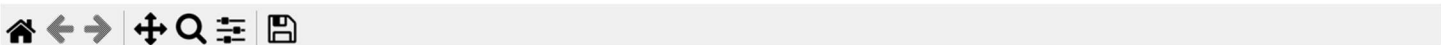
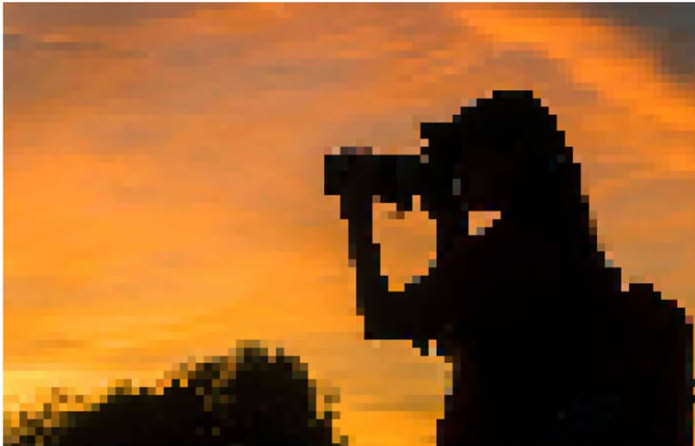
Visual Output

```
--- DOWNSAMPLING ANALYSIS (N=8) ---  
Original Shape:      (444, 691, 3) | Memory: 920,412 bytes  
Downsampled Shape:   (56, 87, 3) | Memory: 14,616 bytes  
Re-expanded Shape:   (444, 691, 3) | Visual: Blocky Pixelation  
Dimension Reduction: 87.39% reduction per axis  
Memory Savings:      98.41% data reduction
```



Original Image

Pixelated (Downsampled N=8 & Re-expanded)



5. Short Answer Deliverable

Question: Why does adding an Alpha channel (RGBA) increase an image array's memory consumption by 33% compared to standard RGB?

Answer:

An uncompressed 8-bit digital image array allocates 1 byte per color channel for every individual pixel (np.uint8):

- **Standard RGB:** Contains 3 channels per pixel (R, G, B), requiring:

$$\text{Memory}_{\text{RGB}} = H \times W \times 3 \times 1 \text{ byte}$$

- **RGBA:** Introduces a 4th channel (Alpha for transparency), requiring:

$$\text{Memory}_{\text{RGBA}} = H \times W \times 4 \times 1 \text{ byte}$$

The fractional increase in memory usage is:

$$\frac{\text{Memory}_{\text{RGBA}} - \text{Memory}_{\text{RGB}}}{\text{Memory}_{\text{RGB}}} = \frac{4 - 3}{3} = \frac{1}{3} \approx 33.33\%$$

Because array size scales linearly with the channel dimension (C) along Axis 2, going from 3 channels to 4 channels directly adds exactly one-third (33.33%) to the total memory footprint.